

ATHENAEUM

Library Management System

Complete Technical Documentation

Source code, project layout, routes, modules, and data flow — explained end to end

BNM Institute of Technology, Bengaluru

Department of Computer Science & Engineering

IPL Course Project — Third Semester

Team: Prithvi G S • Vibhu Manu M S • Vishnu Prasad • Yadhunandan M

Guide: Dr Kavitha Jayaram, Associate Professor, Dept. of CSE

Contents

- 1. Overview & High-Level Architecture
- 2. Project Folder & File Structure
- 3. Backend — server.cpp Deep Dive
- 4. Authentication & Token System
- 5. The Complete REST API Reference
- 6. Data Model & JSON Persistence
- 7. Frontend — React App Structure
- 8. Frontend ↔ Backend Communication
- 9. The Fine Calculation & Overdue Logic
- 10. Dashboard, Stats & Reporting
- 11. UI/UX & the 3D Reading-Room CSS
- 12. Build, Run & Demo Instructions
- 13. Glossary of Key Terms

1. Overview & High-Level Architecture

Athenaeum is a full-stack library management application built as a three-tier system. Each tier has a single clearly-defined responsibility, and the tiers communicate through a small, well-documented HTTP+JSON contract.

The three tiers

- **Presentation tier** — A React 18 single-page application bundled with Vite. Runs in the browser on <http://localhost:5173>. It renders the entire user interface and contains zero business logic.
- **Application tier** — A C++17 HTTP server using `cpp-httplib`. Runs as a standalone binary on <http://localhost:8080>. Handles authentication, validates every request, enforces role-based access, and contains all business rules (loan periods, fine calculation, deletion guards).
- **Persistence tier** — A single JSON file (`library_db.json`) on the server's filesystem. Acts as the entire database. Read and written under a `std::mutex` to keep concurrent requests safe.

Why this split matters

Each tier can be developed, restarted, and tested independently. The frontend developer can mock the API; the backend developer can test endpoints with `curl`; the JSON file can be inspected or edited by hand for debugging. This separation is exactly what the slide on Slide 5 illustrates.

Architectural Principle

The frontend never decides what a user can do. Every authorisation check happens on the backend. If you disable JavaScript in the browser and call the API directly with `curl`, you cannot escalate privileges — the C++ server enforces every rule.

2. Project Folder & File Structure

The repository is organised into a backend folder, a frontend folder, and a few top-level files for documentation and configuration. Below is the typical layout — your actual zip may have minor naming differences.

Typical folder layout

```

athenaeum/
├── backend/
│   ├── server.cpp          ← main backend source (all routes, auth, logic)
│   └── httpplib.h          ← cpp-httpplib single-header HTTP library
├── (vendored)
│   ├── json.hpp            ← nlohmann/json single-header library (vendored)
│   ├── library_db.json     ← the JSON "database" (auto-seeded on first run)
│   ├── Makefile / build.sh ← compilation script (g++ or clang++)
│   └── server              ← compiled binary (after build)
├── frontend/
│   ├── index.html          ← Vite entry HTML
│   ├── package.json        ← npm dependencies (react, react-dom, vite)
│   ├── vite.config.js      ← Vite dev-server config (port 5173, CORS)
│   ├── public/             ← static assets (favicons, images)
│   └── src/
│       ├── main.jsx        ← React root mount
│       ├── App.jsx         ← top-level router & layout
│       ├── api.js           ← all fetch() calls live here
│       ├── auth.js         ← token storage helpers
│       └── components/
│           ├── Landing.jsx  ← 3D bookshelf intro screen
│           ├── Login.jsx    ← login form
│           ├── Register.jsx ← member self-registration
│           ├── Navbar.jsx   ← role-adaptive navigation
│           ├── BookList.jsx ← catalogue grid, search & filter
│           ├── BookForm.jsx ← add / edit book modal (admin)
│           ├── MemberList.jsx ← member management (admin)
│           ├── MemberForm.jsx ← add / edit member modal (admin)
│           ├── LoanList.jsx ← loans view (own for member, all for admin)
│           ├── IssueBook.jsx ← issue-a-loan workflow (admin)
│           ├── Dashboard.jsx ← KPIs, ring gauge, leaderboard
│           └── OverdueStrip.jsx ← pulsing overdue banner
│       └── styles/
│           ├── global.css   ← variables, resets, typography
│           ├── bookshelf.css ← 3D bookshelf transforms
│           └── dashboard.css ← conic-gradient ring gauge
├── README.md               ← run instructions
└── .gitignore

```

Two folders, two processes

During development you open two terminal windows. One runs the backend binary, the other runs the Vite dev server. The slides call this the "2-process setup".

- **Terminal 1 — Backend:** `cd backend && ./server` → listens on port 8080
- **Terminal 2 — Frontend:** `cd frontend && npm run dev` → serves on port 5173

Why the libraries are vendored

cpp-httplib (httplib.h) and nlohmann/json (json.hpp) are both single-header C++ libraries — meaning the entire library is one .hpp file you drop into your project. They are checked into the repo rather than fetched from a package manager. That is what the Slide 6 line "all dependencies are vendored — no internet required at runtime" means: the backend will compile and run on an offline machine.

3. Backend — server.cpp Deep Dive

All backend logic lives in a single C++ file: `server.cpp`. It is structured into five conceptual sections, in order from top to bottom.

3.1 Includes and setup

Top of server.cpp

```
#include "httplib.h"
#include "json.hpp"
#include <fstream>
#include <mutex>
#include <random>
#include <chrono>
#include <unordered_map>

using json = nlohmann::json;
using namespace httplib;

const std::string DB_PATH = "library_db.json";
const int LOAN_PERIOD_DAYS = 14;
const double FINE_PER_DAY = 2.0;           // ₹2 per day overdue

std::mutex db_mutex;                      // protects JSON file I/O
std::unordered_map<std::string, int> tokens; // token → memberId
```

Three constants live at the top so they can be tuned without hunting through code: the database file path, the 14-day loan period, and the daily fine rate. These three constants encode the core business rules.

3.2 Database load / save helpers

Two helper functions wrap all file I/O. Every read and every write goes through these, never directly. The mutex makes them thread-safe — `cpp-httplib` spawns a thread per request, so two simultaneous loan-issuance requests cannot corrupt the file.

Database helpers

```
json load_db() {
    std::lock_guard<std::mutex> lock(db_mutex);
    std::ifstream f(DB_PATH);
    if (!f.good()) return seed_db();    // first run → seed
    json j; f >> j; return j;
}

void save_db(const json& j) {
    std::lock_guard<std::mutex> lock(db_mutex);
    std::ofstream f(DB_PATH);
```

```
f << j.dump(2);           // 2-space indent
}
```

3.3 Auto-seed on first run

When the backend starts and `library_db.json` does not exist, a seed function generates a default admin account, a few sample members, and a starter book catalogue. This is what makes the demo "just work" on a fresh machine — no manual setup.

- **Default admin login:** `admin@bnmit` / `admin123` (verify against your actual seed function)

3.4 Authentication helpers

Two small functions handle token generation and verification. Tokens live only in an in-memory `std::unordered_map` keyed by the token string and returning the member id.

Token helpers

```
std::string make_token() {
    // 32-character hex string from a random engine
    std::mt19937_64 rng(std::random_device{}());
    std::stringstream ss;
    for (int i = 0; i < 4; i++) ss << std::hex << rng();
    return ss.str().substr(0, 32);
}

int verify_token(const Request& req) {
    auto it = req.headers.find("X-Token");
    if (it == req.headers.end()) return -1;
    auto t = tokens.find(it->second);
    return t == tokens.end() ? -1 : t->second; // returns memberId or -1
}
```

Why in-memory tokens?

Persisting tokens to disk would add complexity (expiry, cleanup, race conditions). For an IPL-scope project, the trade-off is acceptable: a server restart logs everyone out, which the README warns users about. This is exactly what Slide 11 means by "Tokens are in-memory only (restart = logout)."

3.5 Route registration and main()

The bottom of `server.cpp` registers every route with `cpp-httplib` and then calls `listen()`. The pattern is the same for every endpoint: parse the request, verify the token (for protected routes), call business logic, send back JSON.

main() and route registration

```
int main() {
```

```

Server svr;

// CORS – required because the frontend runs on a different port
svr.set_default_headers({
    {"Access-Control-Allow-Origin", "http://localhost:5173"},
    {"Access-Control-Allow-Headers", "Content-Type, X-Token"},
    {"Access-Control-Allow-Methods", "GET, POST, PUT, DELETE, OPTIONS"}
});
svr.Options(R("/api/.*)", [](const Request&, Response& res) {
    res.status = 204;    // preflight handler
});

// ----- routes -----
svr.Post("/api/login",      handle_login);
svr.Post("/api/register",   handle_register);
svr.Get ("/api/books",      handle_list_books);
svr.Post("/api/books",      handle_add_book);
svr.Put ("/api/books/:id",   handle_update_book);
svr.Delete("/api/books/:id", handle_delete_book);
// ... members, loans, stats, report
svr.Get ("/api/report/loans.csv", handle_csv_export);

std::cout << "Athenaeum API listening on http://localhost:8080\n";
svr.listen("0.0.0.0", 8080);
}

```

The CORS block is essential: because the frontend runs on port 5173 and the API on 8080, the browser treats them as different origins and would otherwise block the request. The Options handler responds to the preflight check the browser sends before any non-GET request.

4. Authentication & Token System

Athenaeum uses simple token authentication — no JWT, no OAuth, no third-party identity provider. The mechanism is intentionally small so it can be understood and demonstrated end-to-end.

The five-step login flow

- **Step 1** — User submits email and password to `POST /api/login`.
- **Step 2** — Backend looks up the member in `library_db.json` by email and compares the password.
- **Step 3** — If valid, the backend generates a 32-character hex token, stores it in the in-memory map `tokens[token] = memberId`, and returns `{ token, role, name, memberId }` to the client.
- **Step 4** — The React app saves the token (in memory or `sessionStorage`) and stamps every subsequent request with `X-Token: <token>`.
- **Step 5** — On every protected endpoint, `verify_token()` reads the header, looks up the map, and either returns the `memberId` or 401 Unauthorized.

Two roles, two checks

Once the `memberId` is known, the backend fetches the member record and reads the `role` field.

Protected admin endpoints add a second check after token verification — if `role` is not exactly "admin", the handler returns 403 Forbidden.

The auth gate on an admin route

```
int mid = verify_token(req);
if (mid < 0) { res.status = 401; return; } // no/invalid token

json db = load_db();
auto& member = find_member(db, mid);
if (member["role"] != "admin") {
    res.status = 403; // logged in, but wrong role
    res.set_content("{\"error\":\"Forbidden\"}", "application/json");
    return;
}
// ... admin-only logic here
```

Role capability matrix

Capability	Admin	Member
Browse catalogue & search	Yes	Yes
Self-register a new account	—	Yes

Capability	Admin	Member
Add / Edit / Delete books	Yes	No
Add / Edit / Delete members	Yes	No
Issue a loan to a member	Yes	No
Process a return & settle fine	Yes	No
View own loans & fines	Yes	Yes
View all loans (across all members)	Yes	No
Access dashboard stats	Yes	No
Download loan-history CSV	Yes	No

Known limitation: plain-text passwords

Passwords are currently stored as plain text in `library_db.json`. The slide on the data model carries an asterisk acknowledging this. The first item in the Future Scope (Slide 14) is replacing plain text with bcrypt hashing — implementation is straightforward but was descoped for this iteration.

5. The Complete REST API Reference

The backend exposes 15 endpoints across five resource families: auth, books, members, loans, stats, and reports. Every response is JSON unless explicitly noted otherwise (CSV export).

Endpoint summary

Method	Path	Access	Purpose
POST	/api/login	Public	Authenticate, return token
POST	/api/register	Public	Self-register as member
GET	/api/books	Public	List / search / filter books
POST	/api/books	Admin	Add a book to the catalogue
PUT	/api/books/:id	Admin	Update a book or copy count
DELETE	/api/books/:id	Admin	Remove a book
GET	/api/members	Admin	List all members
POST	/api/members	Admin	Admin-add a member
PUT	/api/members/:id	Admin	Update name, role, password
DELETE	/api/members/:id	Admin	Remove a member (guards loans)
GET	/api/loans	Auth	Own loans (member) / all (admin)
POST	/api/loans	Admin	Issue a book to a member
POST	/api/loans/:id/return	Admin	Process return & settle fine
GET	/api/stats	Admin	Dashboard KPIs + leaderboard
GET	/api/report/loans.csv	Admin	Full loan history as CSV

5.1 Authentication endpoints

POST /api/login

Login example

Request:

```
{ "email": "admin@bnmit", "password": "admin123" }
```

```
Response 200:
{ "token": "a1b2c3...32chars", "role": "admin",
  "name": "Default Admin", "memberId": 1 }

Response 401:
{ "error": "Invalid credentials" }
```

POST /api/register

Self-registration is intentionally public — that's how a new student joins the library. The backend forces the role to "member" regardless of what the request body contains, so this endpoint cannot be abused to create an admin.

Register example

```
Request:
{ "name": "Jane Doe", "email": "jane@bnmit", "password": "secret" }

Response 201:
{ "token": "...", "role": "member", "name": "Jane Doe", "memberId": 42 }

Response 409:
{ "error": "Email already registered" }
```

5.2 Book endpoints

GET /api/books

The only book endpoint that is public — anyone can browse the catalogue. Supports optional query parameters for search and filtering:

- `?q=text` — search title or author (case-insensitive substring match)
- `?category=Fiction` — filter by category
- `?available=true` — only show books with at least one available copy

List books example

```
GET /api/books?q=clean&available=true

Response 200:
[ { "id": 7, "title": "Clean Code",
    "author": "Robert C. Martin", "isbn": "978-0132350884",
    "category": "Programming", "total": 3, "available": 2 },
  ... ]
```

POST /api/books (admin)

Add book example

```
Request (X-Token header required):
{ "title":"Sapiens", "author":"Yuval Harari", "isbn":"978-0062316097",
  "category":"History", "total": 4 }

Response 201:
{ "id": 28, "title":"Sapiens", "available": 4, ...the full record... }
```

On insert, the backend sets available = total automatically. The id is assigned by incrementing the highest existing book id.

PUT /api/books/:id (admin)

Updates any subset of the book fields. The handler is careful to recompute available when total changes — if you bump total from 3 to 5 while two copies are on loan, available becomes $5 - 2 = 3$.

DELETE /api/books/:id (admin)

Removes a book. The handler does not currently guard against deletion of a book with active loans — a known small gap, listed informally for future tightening.

5.3 Member endpoints

GET /api/members (admin)

Returns all members with id, name, email, and role. Passwords are stripped from the response — even an admin cannot see another member's password.

DELETE /api/members/:id (admin) — has a guard

This is the only delete endpoint with an explicit safety guard. Before removing the member, the handler scans the loans array for any active loan (`returnedAt == 0`) belonging to that member. If found, the deletion is rejected with 409 Conflict.

Member deletion guard

```
Response 409 (member has active loans):
{ "error": "Cannot delete member with active loans",
  "activeLoans": 2 }
```

5.4 Loan endpoints

GET /api/loans (auth)

The role-sensitive endpoint. The handler verifies the token to get the memberId, then loads the member's role:

- **If admin:** return the entire loans array, joined with member name and book title for display convenience.
- **If member:** filter the loans array to entries where memberId matches, and return only those.

POST /api/loans (admin) — issue a book*Issue a loan*

Request:

```
{ "bookId": 7, "memberId": 42 }
```

Backend logic:

1. Verify `book.available > 0` → else 409
2. Decrement `book.available` by 1
3. Create loan record:


```
issuedAt = current epoch seconds
dueAt    = issuedAt + (14 * 86400)
returnedAt = 0
fine     = 0.0
```
4. Save db, return loan

Response 201:

```
{ "id":105, "bookId":7, "memberId":42,
  "issuedAt":1717564800, "dueAt":1718774400,
  "returnedAt":0, "fine":0.0 }
```

POST /api/loans/:id/return (admin) — process return*Process a return*

Backend logic:

1. Look up loan by id → 404 if not found or already returned
2. `now` = current epoch seconds
3. If `now > dueAt`:


```
days_overdue = ceil((now - dueAt) / 86400)
fine = days_overdue * 2.0
```

 Else:


```
fine = 0.0
```
4. `loan.returnedAt` = `now`
5. `loan.fine` = `fine`
6. `book.available` += 1
7. Save db, return updated loan with fine

5.5 Stats endpoint**GET /api/stats (admin)**

Computes all dashboard numbers on demand — there is no separately stored stats table. The handler walks the books and loans arrays and produces:

- `totalTitles` — number of distinct book records
- `totalCopies` — sum of `book.total` across all books
- `onLoan` — count of loans where `returnedAt == 0`
- `overdue` — count of active loans where `now > dueAt`

- `totalMembers` — number of member records
- `outstandingFine` — sum of computed fines on active overdue loans
- `leaderboard` — top 10 books ranked by all-time borrow count

5.6 Report endpoint

GET /api/report/loans.csv (admin)

Returns a CSV file (Content-Type: text/csv) rather than JSON. The handler iterates the entire loans array, joins each loan with the member name and book title, and writes one row per loan with columns: loanId, memberName, bookTitle, issuedAt, dueAt, returnedAt, fine.

CSV export

Response headers:

Content-Type: text/csv

Content-Disposition: attachment; filename="loans.csv"

Response body:

LoanID,Member,Book,Issued,Due,Returned,Fine

1,Prithvi G S,Clean Code,2025-09-01,2025-09-15,2025-09-14,0.00

2,Jane Doe,Sapiens,2025-09-05,2025-09-19,,8.00

...

6. Data Model & JSON Persistence

Everything the system knows about its users, catalogue, and circulation lives in one JSON file. There are exactly three top-level arrays: books, members, and loans. Below is the shape of each.

6.1 The library_db.json structure

Top-level JSON shape

```
{
  "books": [ { ... } ],
  "members": [ { ... } ],
  "loans": [ { ... } ],
  "nextIds": { "book": 28, "member": 43, "loan": 106 }
}
```

The nextIds object tracks the next id to assign for each entity type. Storing them explicitly is simpler than scanning the arrays for the max id on every insert.

6.2 Book

Book entity

```
{
  "id": 7, // int, auto-increment
  "title": "Clean Code", // string
  "author": "Robert C. Martin", // string
  "isbn": "978-0132350884", // string
  "category": "Programming", // string
  "total": 3, // int, total copies owned
  "available": 2 // int, currently on shelf
}
```

Invariant: $0 \leq \text{available} \leq \text{total}$. Every issue decrements available; every return increments it back.

6.3 Member

Member entity

```
{
  "id": 42, // int, auto-increment
  "name": "Jane Doe", // string
  "email": "jane@bnmit", // string, unique
  "password": "secret", // string *plain text, see warning
  "role": "member" // "admin" or "member"
}
```

6.4 Loan

Loan entity


```

{
  "id": 105,                // int, auto-increment
  "bookId": 7,              // foreign key → Book.id
  "memberId": 42,           // foreign key → Member.id
  "issuedAt": 1717564800,    // epoch seconds, when issued
  "dueAt": 1718774400,       // issuedAt + 14 days
  "returnedAt": 0,          // 0 = still active; else epoch
  "fine": 0.0               // ₹; set when returned
}

```

Why epoch seconds?

Storing dates as integer epoch seconds removes timezone, parsing, and locale ambiguity. The frontend converts them to human-readable dates with `new Date(epoch * 1000).toLocaleDateString()` at render time. The slides refer to these fields as "epoch long" on Slide 11.

6.5 Persistence pattern: read-modify-write

Every mutation follows the same four steps:

- Acquire the `db_mutex` (via `std::lock_guard` inside the `load_db` / `save_db` helpers).
- Load the entire JSON file into memory.
- Mutate the in-memory JSON object.
- Write the entire JSON back to disk and release the lock.

This is intentionally simple — far less performant than a real database for high write volumes, but transparent, debuggable, and crash-safe in the sense that a partially written file is rare (writes are buffered and small).

7. Frontend — React App Structure

The React application is a single-page app built with Vite. There is no router library — navigation is handled with conditional rendering driven by a small state machine in App.jsx.

7.1 main.jsx — the entry point

```
main.jsx
import React from "react";
import { createRoot } from "react-dom/client";
import App from "./App.jsx";
import "./styles/global.css";

createRoot(document.getElementById("root")).render(<App />);
```

7.2 App.jsx — top-level state

App.jsx holds the three pieces of state every other component needs: the token, the current user (role + name), and the current view name. It also handles the 3D landing animation that plays once on first load.

```
App.jsx (simplified)
function App() {
  const [token, setToken] = useState(null);
  const [user, setUser] = useState(null); // {id, name, role}
  const [view, setView] = useState("landing"); // landing|login|books|...

  if (view === "landing") return <Landing onEnter={() => setView("login")} />;
  if (!token) return <Login onLogin={(t,u) => { setToken(t);
setUser(u); setView(u.role === "admin" ? "dashboard" : "books"); }} />;

  return (
    <>
      <Navbar role={user.role} onNav={setView} onLogout={() => { setToken(null);
setUser(null); setView("login"); }} />
      {view === "books" && <BookList token={token} role={user.role} />}
      {view === "members" && <MemberList token={token} />}
      {view === "loans" && <LoanList token={token} role={user.role} />}
      {view === "dashboard" && <Dashboard token={token} />}
    </>
  );
}
```

7.3 api.js — the single source of HTTP truth

Every fetch() call in the app is funnelled through helpers in src/api.js. This keeps the rest of the code free of repeated boilerplate and means changing the backend URL is a one-line edit.

api.js (illustrative)

```

const BASE = "http://localhost:8080";

async function call(path, method, token, body) {
  const headers = { "Content-Type": "application/json" };
  if (token) headers["X-Token"] = token;
  const res = await fetch(BASE + path, {
    method,
    headers,
    body: body ? JSON.stringify(body) : undefined,
  });
  if (!res.ok) throw new Error((await res.json()).error || res.statusText);
  return res.json();
}

export const login      = (email, password) => call("/api/login", "POST",
null, { email, password });
export const register   = (data)           => call("/api/register", "POST",
null, data);
export const listBooks  = (token, q)       => call("/api/books?q="+ (q || ""),
"GET", token);
export const addBook    = (token, b)       => call("/api/books", "POST",
token, b);
export const updateBook = (token, id, b)   => call("/api/books/"+id, "PUT",
token, b);
export const deleteBook = (token, id)      => call("/api/books/"+id, "DELETE",
token);
export const listMembers = (token)         => call("/api/members", "GET",
token);
export const listLoans  = (token)         => call("/api/loans", "GET",
token);
export const issueLoan  = (token, bookId, memberId) => call("/api/loans",
"POST", token, { bookId, memberId });
export const returnLoan = (token, id)      => call("/api/loans/"+id+"/return",
"POST", token);
export const fetchStats = (token)         => call("/api/stats", "GET",
token);

```

7.4 Component responsibilities

Component	Responsibility
Landing.jsx	Plays the 3D bookshelf intro animation on first load.
Login.jsx	Login form. Calls <code>api.login()</code> and lifts token/user up.
Register.jsx	Self-registration form for new members.

Component	Responsibility
Navbar.jsx	Renders different links depending on role.
BookList.jsx	Catalogue grid. Owns the live search input and category filter.
BookForm.jsx	Modal for add/edit. Used by admin only.
MemberList.jsx	Admin-only member CRUD.
LoanList.jsx	Shows own loans or all loans depending on role.
IssueBook.jsx	Admin form: pick a member, pick a book, click Issue.
Dashboard.jsx	Calls /api/stats; renders KPIs, leaderboard, ring gauge.
OverdueStrip.jsx	Pulsing alert banner shown when overdue > 0.

8. Frontend ↔ Backend Communication

All communication is plain HTTP+JSON. There is no WebSocket, no GraphQL, no gRPC. Below is the full lifecycle of a single user action — "admin returns a book" — traced from button click to UI update.

Step-by-step trace

- **1.** Admin clicks the green "Return" button next to a loan in `LoanList.jsx`.
- **2.** The component calls `api.returnLoan(token, loanId)`.
- **3.** `api.js` builds `fetch('http://localhost:8080/api/loans/105/return', { method: 'POST', headers: { 'X-Token': '...' } })`.
- **4.** Browser sends a CORS preflight (OPTIONS) — backend replies 204.
- **5.** Browser sends the actual POST — `cpp-httpplib` routes it to `handle_return_loan`.
- **6.** Backend: `verify_token` → check admin role → `load_db` → find loan → compute fine → update loan + `book.available` → `save_db`.
- **7.** Backend returns 200 with the updated loan JSON.
- **8.** `api.js` resolves the promise. `LoanList` re-fetches the full loans list and re-renders. UI shows the loan as Returned with the final fine.

CORS — why it matters and how it's configured

The browser blocks cross-origin requests by default. Because the frontend (5173) and backend (8080) are different origins, the backend must explicitly opt-in. Three headers do the work, set in `main()` at the top of every response:

- `Access-Control-Allow-Origin: http://localhost:5173`
- `Access-Control-Allow-Headers: Content-Type, X-Token`
- `Access-Control-Allow-Methods: GET, POST, PUT, DELETE, OPTIONS`

If the guide asks why the OPTIONS handler is needed: any non-trivial request (one with a custom header like `X-Token`, or a non-GET method) triggers a preflight — the browser first sends OPTIONS asking "is this allowed?" before sending the real request. The backend must reply 204 No Content with the allow headers.

9. The Fine Calculation & Overdue Logic

Fines are computed live, not stored. The stored fine field on a loan is only set when the loan is actually returned. For active overdue loans, the dashboard computes the current accruing fine on every stats request.

9.1 The formula

Fine formula (server.cpp)

```
days_overdue = ceil((now - dueAt) / 86400)    // 86400 = seconds in a day
fine          = days_overdue * FINE_PER_DAY    // FINE_PER_DAY = ₹2.00
```

- If $\text{now} \leq \text{dueAt}$, `days_overdue` is zero and fine is zero.
- The `ceil()` ensures a loan one second past midnight on day 15 already counts as one overdue day — the borrower cannot game partial days.
- The 2.0 constant lives at the top of `server.cpp` as `FINE_PER_DAY`. Change it once, recompile, done.

9.2 Where the fine is calculated

Two places compute fines, with a deliberate difference:

- **On return** — In `handle_return_loan`. Calculated once, written into `loan.fine`, and persisted. This is the final settled amount.
- **On stats request** — In `handle_stats` and in any loan-listing path. Computed for each active overdue loan to show the live accruing amount. Never written to disk.

9.3 Worked examples

Issued	Due	Returned	Days Overdue	Fine
Sep 1	Sep 15	Sep 12	0	₹ 0.00
Sep 1	Sep 15	Sep 15	0	₹ 0.00
Sep 1	Sep 15	Sep 16	1	₹ 2.00
Sep 1	Sep 15	Sep 20	5	₹ 10.00
Sep 1	Sep 15	still active (today: Sep 25)	10 (live)	₹ 20.00 (live)

10. Dashboard, Stats & Reporting

The dashboard is the single most visually striking screen and the one likely to draw the most questions. It is a pure consumer of GET /api/stats — no separate logic on the frontend except presentation.

10.1 The six KPI tiles

Tile	Source field	Computation
Total Titles	totalTitles	books.length
Total Copies	totalCopies	sum of book.total
On Loan	onLoan	count of loans where returnedAt == 0
Overdue	overdue	count of active loans where now > dueAt
Members	totalMembers	members.length
Outstanding Fine	outstandingFine	sum of live fines on overdue loans

10.2 The Most-Borrowed leaderboard

The backend walks the entire loans array and counts how many times each bookId appears. The result is sorted descending and the top 10 are returned with their book title and total borrow count. The top three get gold/silver/bronze medallions on the frontend — purely CSS, no extra data.

10.3 The collection-health ring gauge

This is the green ring you see on the dashboard. It is drawn with a single CSS conic-gradient — no chart library, no canvas, no SVG. The angle is computed from the ratio of available copies to total copies.

Ring gauge CSS

```
.health-ring {
  width: 180px; height: 180px;
  border-radius: 50%;
  background: conic-gradient(
    #2C5F2D 0deg calc(var(--pct) * 3.6deg), /* available – green arc */
    #E0E0E0 calc(var(--pct) * 3.6deg) 360deg /* on-loan – grey arc */
  );
  animation: spin-in 0.8s ease-out;
}
```

The --pct CSS variable is set inline by React based on stats.totalCopies and stats.onLoan. Multiplying by 3.6 converts percentage to degrees (since $360^\circ \div 100\% = 3.6^\circ/\%$).

10.4 The overdue spotlight strip

Shown only when `stats.overdue > 0`. Listed for every active overdue loan: member name, book title, days overdue, and the live accruing fine. A CSS `@keyframes` rule pulses the background colour gently to draw attention without being obnoxious.

10.5 CSV export

Pressing the "Download CSV" button on the dashboard hits `GET /api/report/loans.csv`. The browser saves the file (because the response carries `Content-Disposition: attachment`). The CSV columns match the loans table fields plus joined-in member name and book title — useful for opening in Excel or running quick pivots outside the system.

11. UI/UX & the 3D Reading-Room CSS

The interface has a deliberately tactile, library-themed aesthetic. Crucially, every effect is plain CSS — no Three.js, no WebGL, no GLSL shaders. This keeps the bundle tiny and the project portable.

11.1 The landing-page 3D bookshelf

On first load, the user sees an animated bookshelf with leather-spined books before the login screen. The illusion is built from layered CSS:

- A wood-grain bookshelf background using repeating linear-gradients.
- Individual book spines as div elements with CSS 3D transforms (`translateZ`, `rotateY`) to give depth.
- Brass fittings and shelf edges via box-shadow with multiple values to fake highlights and lighting.
- A perspective container with `perspective: 1200px` so child transforms render with realistic foreshortening.

11.2 Role-adaptive navigation

The Navbar component receives a `role` prop and renders different link sets accordingly:

Navbar.jsx (illustrative)

```
function Navbar({ role, onNav, onLogout }) {
  const adminLinks = ["Catalogue", "Members", "Loans", "Dashboard"];
  const memberLinks = ["Catalogue", "My Loans"];
  const links = role === "admin" ? adminLinks : memberLinks;
  return (
    <nav>
      {links.map(l => <button onClick={() =>
onNav(l.toLowerCase())}>{l}</button>)}
      <button onClick={onLogout}>Logout</button>
    </nav>
  );
}
```

11.3 Loan status colour coding

Status	Colour	Visual rule
Active	Blue	Calm, default — "all is well"
Overdue	Red, with pulse	Pulses opacity 0.6 ↔ 1.0 to attract attention
Returned	Muted green	Completed, no further action needed

11.4 Responsive card grids

Books, members, and loans render as card grids using CSS Grid with `grid-template-columns: repeat(auto-fill, minmax(280px, 1fr))`. This means cards reflow into 1, 2, 3, or 4 columns depending on viewport width with zero JavaScript.

11.5 Live client-side search

All filtering — text search, category filter, availability toggle — happens in React state on the browser. Once the catalogue is fetched once, filtering is instant: no round-trip to the backend per keystroke. This is what the slides call out as a UX win.

12. Build, Run & Demo Instructions

12.1 Prerequisites

- g++ 9+ or clang++ 10+ (any C++17-capable compiler).
- Node.js 18+ and npm.
- A modern browser (Chrome, Edge, Firefox, Safari).

12.2 Build the backend

Build command

```
cd backend
g++ -std=c++17 -O2 -pthread server.cpp -o server
# (or: clang++ -std=c++17 -O2 -pthread server.cpp -o server)
```

- `-std=c++17` — required, cpp-httplib uses C++17 features.
- `-O2` — optimisation level for a smooth demo.
- `-pthread` — required because cpp-httplib spawns threads.

12.3 Run the backend

Start backend

```
./server
# Output:  Athenaueum API listening on http://localhost:8080
```

12.4 Install and run the frontend

Start frontend

```
cd ../frontend
npm install          # only on first run
npm run dev
# Output:  VITE ready in 320 ms
#          Local:  http://localhost:5173/
```

12.5 Open the app

Open <http://localhost:5173> in a browser. The 3D landing screen plays once, then you reach the login page. Use the seeded admin account (check your seed function for exact credentials) to demonstrate admin flows; use Register to demonstrate member self-signup.

12.6 Demo checklist for the guide

- Pre-start both processes before the meeting. A cold start with build errors during the demo is unrecoverable.

- Open `library_db.json` in a side editor — being able to show the data persisting in real time is a strong moment.
- Have one overdue loan ready (set `issuedAt` back by ~20 days in the JSON) so you can demonstrate the pulsing strip and fine.
- Have curl or a REST client ready to hit `/api/loans` with no token, to show a 401 response and prove the server-side enforcement.

13. Glossary of Key Terms

Term	Meaning in this project
REST	Architectural style where each URL is a resource and HTTP methods (GET/POST/PUT/DELETE) describe the action.
JSON	JavaScript Object Notation — the wire format for every request and response, and the disk format for the database.
Token	A 32-character hex string that proves a user is logged in. Sent in the X-Token header on every protected request.
CORS	Cross-Origin Resource Sharing — the browser security model the backend must opt into so the frontend on port 5173 can talk to it on port 8080.
Mutex	A mutual-exclusion lock. <code>std::mutex</code> serialises file access so two threads cannot corrupt <code>library_db.json</code> .
Epoch	Seconds elapsed since 1 Jan 1970 UTC. All timestamps are stored this way for simplicity.
Vite	Modern frontend build tool. Provides hot-module-reload during dev and bundles for production.
JSX	JavaScript with embedded HTML-like syntax. Compiled to plain JS by Vite before the browser sees it.
conic-gradient	CSS function that produces a circular sweep of colour. Used to draw the ring gauge with zero JS.
Preflight	The OPTIONS request a browser sends before a non-trivial cross-origin request, to check it is allowed.
Seeding	Auto-creating a default dataset on first run so the system is immediately usable.
Idempotent	An operation that produces the same result whether called once or many times — GET is idempotent, POST usually is not.

Final note

This document describes the Athenaeum codebase based on the design captured in the project presentation. Variable names, helper function signatures, and minor structural details in the code may differ slightly from what is shown here — the slides did not include the source. Treat the code snippets in this document as faithful illustrations of the patterns used, not as a verbatim copy of `server.cpp`.